Deep Learning Operations (DL-Ops)
Assignment – 1

Name: Kunal Mishra
Roll Number: M25CSA036
Course: DL-Ops
Date: 24/01/2026

Colab Notebook Link:
https://colab.research.google.com/drive/1KvirxsQvSsGZPJsTIeeCR7-_I7cJIVzW?usp=sharing

GitHub Repository:
https://github.com/Givhel/MLOps_KunalMishra_M25CSA036

GitHub Pages:
https://givhel.github.io/MLOps_KunalMishra_M25CSA036/

1. Introduction

In this assignment, we study the training and performance analysis of deep learning models and classical machine learning models on image classification tasks. The MNIST and FashionMNIST datasets are used to evaluate the performance of ResNet architectures (ResNet-18 and ResNet-50) as well as Support Vector Machines (SVM).

The objectives of this assignment are:

1. To analyze the classification performance of ResNet models on MNIST and FashionMNIST datasets.
2. To study the effect of different hyperparameters such as batch size, optimizer, and learning rate.
3. To compare classical machine learning (SVM) with deep learning models.
4. To compare CPU and GPU performance in terms of training time, FLOPs, and accuracy.
5. To understand the impact of Automatic Mixed Precision (AMP) on GPU training.

All experiments were implemented using Python and PyTorch. The complete code and executed experiments are available in the shared Google Colab notebook.

2. Dataset Description

Two standard benchmark datasets were used in this assignment:

1. MNIST Dataset:
   - Consists of grayscale images of handwritten digits from 0 to 9.
   - Image size: 28 × 28 pixels.
   - Number of classes: 10.
   - Total samples: 60,000 training images and 10,000 test images.

2. FashionMNIST Dataset:
   - Consists of grayscale images of clothing items such as shirts, shoes,coats, etc.
   - Image size: 28 × 28 pixels.
   - Number of classes: 10.
   - Total samples: 60,000 training images and 10,000 test images.

For both datasets, a 70% – 10% – 20% split was used:
- 70% for training
- 10% for validation
- 20% for testing

The datasets were normalized and loaded using PyTorch's torchvision utilities.

3. Model Description

In this assignment, two deep learning models from the ResNet family were used:

1. ResNet-18:
   - A relatively lightweight convolutional neural network.
   - Contains 18 layers with residual (skip) connections.
   - Suitable for fast training and lower computational cost.

2. ResNet-50:
   - A deeper and more complex architecture with 50 layers.
   - Has a higher number of parameters and computational requirements.
   - Capable of learning more complex features.

Both models were initialized with pretrained = False, meaning that the weights were randomly initialized and no pre-trained ImageNet weights were used.

The first convolutional layer was modified to accept single-channel (grayscale) images, and the final fully connected layer was modified to output 10 classes corresponding to the datasets.

For optimization:
- SGD and Adam optimizers were used.
- Different batch sizes and learning rates were tested.

4. Q1(a): CNN Experiments on MNIST and FashionMNIST

4.1 MNIST Results

			Test Classification Accuracy (in %)	
Batch Size	**Optimizers**	**Learning Rate**	**ResNet-18**	**ResNet-50**
16	SGD	0.001	98.37	97.94
16	SGD	0.0001	95.07	86.62
16	Adam	0.001	98.18	97.94
16	Adam	0.001	98.65	96.72
32	SGD	0.001	97.53	96.55
32 (Optional)	SGD	0.0001	92.99	__
32 (Optional)	Adam	0.001	98.91	__
32	Adam	0.0001	92.99	96.78

4.2 FashionMNIST Results

			Test Classification Accuracy (in %)	
Batch Size	**Optimizers**	**Learning Rate**	**ResNet-18**	**ResNet-50**
16	SGD	0.001	88.18	84.71
16	SGD	0.0001	82.25	77.39
16	Adam	0.001	88.35	87.18
16	Adam	0.001	82.76	87.92
32	SGD	0.001	87.22	83.83
32 (Optional)	SGD	0.0001	_	__
32 (Optional)	Adam	0.001	_	__
32	Adam	0.0001	90.33	86.24

5. Q1(b): SVM Experiments

In this section, Support Vector Machines (SVM) were trained on both MNIST and FashionMNIST datasets using polynomial (poly) and radial basis function (rbf) kernels. The goal was to compare the performance of a classical machine learning model with deep learning models in terms of classification accuracy and training time.

Dataset	**Kernel**	**Test Accuracy(%)**	**Training Time(ms)**
MNIST	poly	98.09	97204.21
MNIST	RBF	97.86	122653.03
FashionMNIST	Poly	89.22	139078.01
FashionMNIST	RBF	88.60	155279.37

The SVM model achieved very high accuracy on the MNIST dataset, reaching above 98% using the polynomial kernel. This shows that classical machine learning models perform extremely well on simple and well-structured datasets.

However, for FashionMNIST, the accuracy dropped significantly to around 88–89%. This indicates that SVM struggles with more complex visual patterns compared to deep learning models. The RBF kernel required higher training time compared to the polynomial kernel due to its computational complexity.

Compared to CNN models, SVM is much faster to train but lacks the feature learning capability of deep neural networks, which becomes critical for complex datasets like FashionMNIST.

6. Q2: CPU vs GPU Performance Analysis on FashionMNIST

In this experiment, the performance of deep learning models was analyzed on both CPU and GPU using the FashionMNIST dataset. The objective was to compare training time, computational cost (FLOPs), and classification accuracy for different models and optimizers. All GPU experiments were performed using Automatic Mixed Precision (AMP) to improve computational efficiency. AMP is not applicable for CPU training.

Compute	Model	Optimizer	Epochs	Accuracy (%)	Time(ms)	FLOPs (GFLOPs)
CPU	ResNet-18	SGD	3	86.93	1851212.34	0.96
CPU	ResNet-18	Adam	3	88.08	2748829.62	0.96
CPU	ResNet-50	SGD	1	75.86	1395364.84	2.56
CPU	ResNet-50	Adam	1	70.91	1878685.74	2.56
GPU	ResNet-18	SGD	5	88.34	202796.74	0.96
GPU	ResNet-18	Adam	5	90.34	244300.73	0.96
GPU	ResNet-50	SGD	5	84.39	401587.10	2.56
GPU	ResNet-50	Adam	5	81.62	499694.68	2.56

FLOPs were computed using the THOP library. The reported FLOPs correspond to training computation per batch (forward and backward pass) for batch size = 16. FLOPs depend only on the model architecture and input size, and therefore remain constant across CPU and GPU executions.

From the results, it is evident that GPU execution provides a massive speedup compared to CPU. Although FLOPs remain the same for CPU and GPU, training time is drastically lower on GPU due to parallel processing and

hardware acceleration. For example, ResNet-18 required more than 30 minutes on CPU, while the same model trained in a few minutes on GPU.

ResNet-50 consistently required more training time than ResNet-18 because of its higher FLOPs and deeper architecture. Adam optimizer generally achieved better accuracy but required more training time than SGD.

The use of Automatic Mixed Precision (AMP) further improved GPU efficiency by reducing memory usage and accelerating computations without significantly affecting accuracy. These results clearly demonstrate that GPUs are essential for efficient deep learning training.

7. Conclusion

In this assignment, deep learning and classical machine learning models were evaluated on MNIST and FashionMNIST datasets using various hyperparameters and computational settings. ResNet-18 and ResNet-50 were trained from scratch and their performance was compared in terms of accuracy, training time, and computational cost.

The experiments showed that MNIST is an easy dataset where both models achieve very high accuracy, while FashionMNIST is more challenging and requires stronger feature learning. ResNet-18 provided a good balance between performance and computational efficiency, often performing comparably to ResNet-50 while requiring significantly less training time.

SVM models performed extremely well on MNIST but struggled on FashionMNIST compared to CNNs, highlighting the importance of deep learning for complex image classification tasks.

The CPU vs GPU comparison clearly demonstrated that GPUs are essential for training deep neural networks efficiently. Even though FLOPs remain constant for a given model, GPU execution drastically reduced training time due to parallel computation and hardware acceleration. The use of Automatic Mixed Precision (AMP) further improved GPU performance without significantly affecting accuracy.

Overall, this assignment provided strong practical insights into model performance, hyperparameter tuning, computational efficiency, and the importance of modern hardware accelerators in deep learning.